

HW4_Model_Validation_Report BY GROUP2

代码稳定性 (Code Stability) (赵芳誉)

整体 Notebook 结构清晰、依赖规范，管道从数据读取→特征工程→多模型训练→结果导出能稳定跑通；关键自定义函数（如目标编码与地理特征构造）在常见边界条件下也能返回可用结果，具备复现实验与复跑的可靠性。

该组在模型选择与调参上已经搭建了完善的 GridSearchCV 与自定义评估函数，能系统地对 Lasso/Ridge/ElasticNet 等线性模型做超参搜索，并给出验证集与测试集指标，整体流程完整可复现。但是当前做法存在**同一批数据既用于超参选择又用于报告交叉验证分数**的轻微乐观偏差；并且拟合在 $\log(y)$ 空间而评估口径未完全统一（部分指标混用了对数与原值）。这会线性正则化模型的泛化误差略被低估，横向比较不同模型/特征版本时也不够可比。

Suggestion1：改进切入点：GridSearchCV 外层复用同一折分做评估；自定义 evaluate_model 在对数空间与原空间之间切换不一致（见 evaluate_model 与各模型的训练/预测调用处）。

现状：同一层 CV 既挑超参又汇报“CV 表现”

```
gs = GridSearchCV(model, param_grid, cv=5, scoring="neg_mean_absolute_error")
gs.fit(X_train, np.log(y_train))
cv_score = cross_val_score(gs.best_estimator_, X_train, np.log(y_train), cv=5,
scoring="neg_mean_absolute_error")
```

评估时的口径：有时在 $\log$ 空间，有时 $\exp$ 回原空间再算

建议修复（嵌套交叉验证 + 指标口径统一到“原始金额”）：外层 KFold 仅用于泛化评估；内层 GridSearchCV 仅用于超参选择。

代码运行优化：地理距离计算。该组正确地基于城市中心点构造了“到城市中心距离”等地理特征，增强了可解释性与模型信号；思路清晰、实现直观。但是目前逐行使用 geopy.distance.geodesic 或 DataFrame.apply 计算样本点到中心点的距离，在这个样本量下显著**拖慢训练迭代**；同时不同环境下 geodesic 的计算开销波动较大，影响整体稳定性与复现实验用时。

Suggestion2 改进切入点：compute_city_center_and_distances 及其调用处，采用 df.apply(lambda r: geodesic((r['lat'], r['lon']), (r['clat'], r['clon'])).km, axis=1) 的逐行实现。

现状（低效实现示意）

```
from geopy.distance import geodesic
df["dist_to_center_km"] = df.apply(
    lambda r: geodesic((r["lat"], r["lon"]), (r["center_lat"], r["center_lon"])).km, axis=1
)
```

建议修复（NumPy 矢量化 Haversine，一次性全列计算）：

- 先把每个城市的中心点（已在训练集上计算）merge 回样本；
- 用矢量化 **Haversine** 公式在 NumPy 层批量计算，通常能快一个数量级以上；
- 若还有地理聚类（如 KMeans），确保只在训练集 fit，在验证/测试上 predict，避免重复拟合或泄漏。

```
import numpy as np
```

```
def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371.0 # 地球半径 (km)
```

```

lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
dlat, dlon = lat2 - lat1, lon2 - lon1
a = np.sin(dlat/2.0)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2.0)**2
return 2 * R * np.arcsin(np.sqrt(a))

# 假设 df 已含列: lat, lon, center_lat, center_lon (中心点用训练集统计值)
df["dist_to_center_km"] = haversine_km(
    df["lat"].to_numpy(), df["lon"].to_numpy(),
    df["center_lat"].to_numpy(), df["center_lon"].to_numpy()
)

```

数据处理：（李冰滢）

问题：存在比较严重的数据泄露问题，训练集和测试集的数值型变量有混合插值问题，中位数的来源包含了 test。

建议 3：只使用训练集的中位数进行填补，还可以根据地理位置分层填补。

问题：对多个文本特征列进行重复编码，特征存在严重的共线性问题，有过拟合风险。如：八个朝向的布尔特征、朝向得分、南北通透。

建议 4：根据需要选择编码模式；在进行特征工程之后、模型拟合之前进行 VIF 检验，删掉自变量。

其他建议：特征缺失比例较小的情况下可以使用众数分层填充，其次再用未知填充。

样本选择与数据泄露 (Sample Choice & Data Leakage) （王子卿）

该组在样本选择和数据泄露方面完成出色，正确划分了训练集和验证集的基础上，在处理高基数类别特征（如区域组合键）时，采用了 K-Fold Target Encoding，仅利用了训练集中其他 Fold 的数据进行均值计算，并没有让该样本自身的目标值参与运算。对于测试集，也使用了全量训练集的统计量。避免了目标变量的信息泄露。。

但是在填充缺失值时，运用了全局的中位数/众数填充，造成了一定的统计特征泄露。具体而言，项目先合并了全量数据，然后进行特征工程构建。在对基础数值型特征（如建筑年代、容积率、停车位、物业费等）进行缺失值填充时，代码直接在合并后的全量数据上计算了中位数或众数，并用这些统计值填充了缺失项。这导致测试集的分布特征泄露到了训练集中，违反了“训练过程应对测试数据完全盲目”的原则。

错误位置：Cell18-20

```

# 错误: df_price 包含测试集, median() 计算混入了测试集信息
#处理建筑年代列 - 计算建筑年龄 + 分箱 + 交互特征
median_year = int(df_price['建筑年代_parsed'].median())
df_price['建筑年代_parsed'] = df_price['建筑年代_parsed'].fillna(median_year)

#处理容积率列 - 中位数填充 + 分箱
median_far = df_price['容积率'].median()

```

```
far_filled = df_price['容积率'].fillna(median_far)

#处理停车位列 - 数值化
median_parking = df_price['停车位_num'].median() if df_price['停车位_num'].notna().any()
else 0
X['停车位'] = df_price['停车位_num'].fillna(median_parking)
```